MySQL Midpoint Insertion Strategy & Buffer Pool

Source: Arpit Bhayani Dissection • InnoDB Engine
Focus: Scan Resistance, LRU Sublists & Cache Pollution

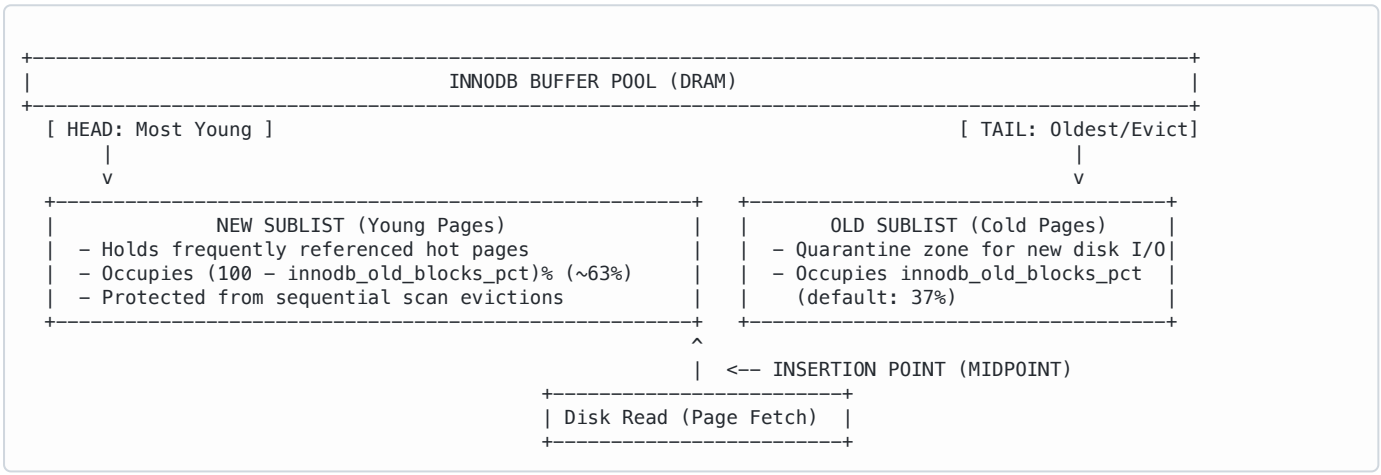
1. The Naive LRU & The Hazard of Cache Pollution

In relational database engines like MySQL InnoDB, DRAM memory access (~50–100 ns) is three to four orders of magnitude faster than NVMe SSD access (~50–100 µs). The **Buffer Pool** is InnoDB's in-memory page cache (defaulting to 16 KB pages) holding table rows, indexes, and undo logs. In a naive Least Recently Used (LRU) cache, every newly fetched page from disk is placed at the *head* of a single doubly linked list, and the oldest page at the *tail* is evicted.

The Problem: When a background batch reporting job, analytical query, or backup (`mysqldump`) triggers a full table scan, it reads millions of cold pages sequentially. Under naive LRU, these one-off pages flood the head of the cache, displacing all the genuinely hot transactional working set (user profiles, active cart sessions, auth tokens). Cache hit rates collapse from 99.9% to below 60%, disk I/O saturates, and database latency explodes. This failure mode is known as **Cache Pollution** or **Cache Thrashing**.

2. Architecture of the Scan-Resistant Buffer Pool

To make the Buffer Pool scan-resistant, InnoDB bisects the LRU list into two dynamic sublists separated by a designated **Midpoint**:



> **The Midpoint Insertion Axiom:** Pages read from disk are *never* placed at the head of the LRU list. They are placed at the **head of the Old Sublist (the Midpoint)**. If the page is part of a one-off table scan and never accessed again, it travels down the small Old Sublist and drops off the tail without ever displacing the hot pages in the New Sublist.

3. Buffer Pool Concurrent Lists

InnoDB maintains three coordinated lists across all allocated page descriptors (`buf_page_t`):

- **Free List:** Pre-allocated, empty frames ready to receive incoming page reads from disk.
- **LRU List:** Partitioned into New and Old sublists, tracking page access recency for cache replacement.
- **Flush List:** Tracks dirty modified pages ordered by oldest unwritten LSN for asynchronous checkpointing to disk.

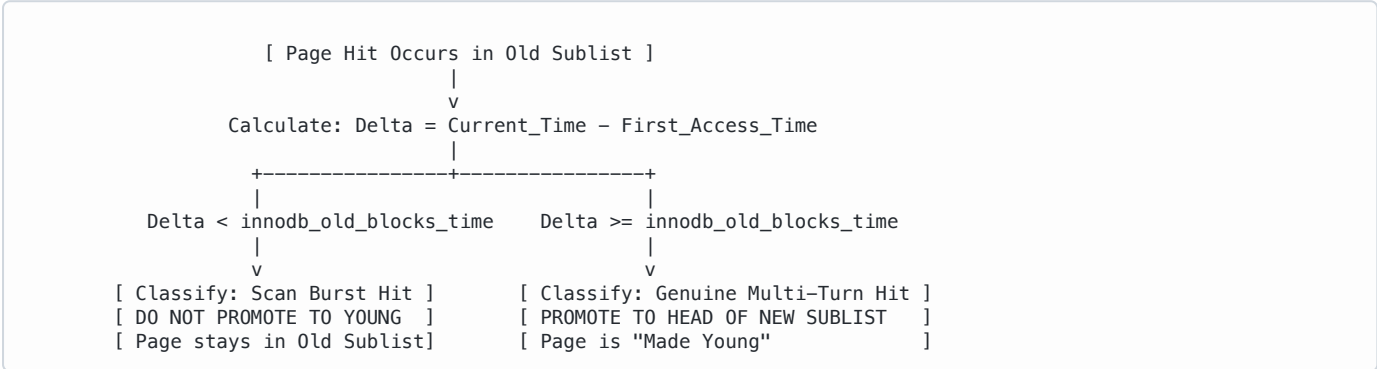
4. The Flaw in Naive Midpoint: Multi-Row Scans

A 16 KB InnoDB page typically stores between 50 and 500 records. When a query performs a sequential scan:

1. Row 1 on Page P is requested → Page P is fetched from disk and inserted at the Midpoint.
2. Microseconds later, the query evaluates Row 2 on Page P → this is an in-memory cache hit!
3. If InnoDB promoted pages to the New Sublist on *any* cache hit, Row 2 would immediately promote Page P to the head of the Young Sublist, completely bypassing the midpoint quarantine!

5. The Temporal Solution: `innodb_old_blocks_time`

To prevent sequential table scans from spoofing the promotion heuristic, InnoDB stamps each page with an access timestamp when it enters the Old Sublist. MySQL introduces a mandatory time gate:



> **Time Window Mechanism:** The default parameter `innodb_old_blocks_time = 1000` requires that a page must reside in the Old Sublist for at least 1,000 milliseconds (1 second) before a subsequent read is allowed to promote it to the New Sublist. All subsequent accesses within that 1-second burst are treated as part of the same scan.

6. Asynchronous Read-Ahead Engines & Midpoint Safety

InnoDB optimizes sequential I/O using asynchronous background prefetchers:

Engine	Trigger Heuristic	Prefetch Action	Default Config
Linear Read-Ahead	Detects that `innodb_read_ahead_threshold` contiguous pages in an extent (64 pages = 1 MB) were read sequentially.	Asynchronously reads the *entire subsequent extent* into the buffer pool.	`innodb_read_ahead_threshold = 56`
Random Read-Ahead	Detects 13 contiguous pages of the same extent already present in the buffer pool.	Asynchronously issues I/O requests for the remainder of the extent.	`innodb_random_read_ahead = OFF`

Why Midpoint Insertion Makes Read-Ahead Safe: In traditional caches, aggressive speculative prefetching risks ejecting active hot pages. In InnoDB, all read-ahead pages are inserted into the *Old Sublist*. If the query does not actually need the prefetched pages, they age out harmlessly at the tail without polluting the hot working set.

7. Production Configuration & Tuning Parameters

Optimal tuning of the Buffer Pool depends on the workload modality (Pure OLTP vs. Mixed OLAP/OLTP):

Parameter	Default	High-Throughput OLTP	Mixed Reporting / Batch	Operational Impact
`innodb_old_blocks_pct`	37 (37%)	`20` (20%)	`40` - `50` (40-50%)	Lowering reserves more memory (80%) for the hot working set; raising broadens scan absorption capacity.
`innodb_old_blocks_time`	`1000` ms	`1000` ms	`2000` - `5000` ms	Extends quarantine duration against slow analytical queries that span several seconds per table.
`innodb_buffer_pool_instances`	1 or 8	8 - 16	8 - 16	Splits buffer pool into independent instances to eliminate mutex lock contention on high CPU cores.

8. Monitoring Buffer Pool Health in Production

Engineers can diagnose buffer pool efficiency and verify scan resistance using `SHOW ENGINE INNODB STATUS`:

```
mysql> SHOW ENGINE INNODB STATUS\G
----------------------
BUFFER POOL AND MEMORY
----------------------
Total large memory allocated 68719476736   -- 64 GB Pool
Buffer pool size        4194304           -- Total 16KB Pages
Free buffers            1024              -- Free Frame Count
Database pages          4090000
Old database pages      1513300           -- Exactly 37% in Old Sublist
Pages made young        8940120           -- Promoted to New Sublist
Pages NOT made young    4910294812        -- Quarantined sequential scan reads!
Buffer pool hit rate    999 / 1000        -- 99.9% Cache Hit Ratio maintained
```

> **Health Metric Rule of Thumb:** In healthy production environments with analytical or backup traffic, the ratio of `Pages NOT made young` should be significantly higher than `Pages made young`. This mathematically proves that table scans are being quarantined and dropped off the tail without corrupting the hot set.

9. First-Principles System Design Summary

- **Decouple Cache Size from Admission Priority:** Do not assume all disk reads deserve equal placement. Admission control (midpoint insertion) separates the right to enter memory from the right to displace hot memory.
- **Spatial Locality vs Temporal Value:** Multiple row reads on a single 16 KB page represent spatial grouping, not temporal durability. A time barrier ($t \ge 1000 ext{ms}$) distinguishes single-pass loops from recurring access patterns.
- **Partitioning Eliminates Cascading Evictions:** By isolating cold scan data to 37% of the buffer pool, the 63% hot working set remains immune to unpredictable query behavior.